A Machine Learning Framework for B2B Price Optimization: Determining Stretch and Floor Prices

Part 1: Strategic Foundations of Algorithmic B2B Pricing

The transition from traditional, often manual, pricing strategies to a sophisticated, data-driven framework represents a pivotal strategic decision for any B2B retail organization. Success in this domain is not merely a matter of deploying the latest machine learning algorithms; it requires a deep, foundational understanding of the unique economic principles governing B2B transactions and an unwavering commitment to a causal inference framework. This section establishes these essential strategic pillars, moving beyond simplistic economic theories to address the nuanced realities of B2B markets and laying the groundwork for the advanced modeling techniques that follow. It will demonstrate that a robust pricing system must be built upon a clear-eyed view of profit optimization, a granular understanding of B2B-specific demand dynamics, and a rigorous methodology for isolating the true causal effect of price on customer behavior.

1.1 The Economics of Price Elasticity in B2B Retail

The fundamental objective of any commercial pricing strategy is to navigate the intricate relationship between price, demand, and cost to achieve a desired business outcome, most commonly the maximization of profit. While the concept of price elasticity of demand—the measure of how quantity demanded responds to a change in price—is central, its application in a real-world B2B context requires a level of sophistication far beyond textbook definitions.

The Profit Optimization Curve

The relationship between price and quantity is classically represented by a downward-sloping demand curve: as price increases, demand decreases. However, the ultimate business objective is not to understand demand for its own sake, but to optimize profit. The relationship between price and profit is distinctly non-linear, typically forming a semicircle or parabolic curve.[1] At the extremes, profit is null. A price of zero may generate maximum demand, but it yields zero revenue and negative profit. Conversely, an infinitely high price yields a very high per-unit profit margin, but with zero units sold, total profit is again null.[1] The goal of a price optimization model is therefore to identify the peak of this profit curve—the price that perfectly balances margin and volume to generate maximum total profit. This peak corresponds to the 'stretch price' that a business can command.

Beyond Unitary Elasticity: The Need for Demand Curve Estimation

A single price elasticity coefficient, such as an elasticity of -1.25 indicating a 1.25% drop in demand for every 1% increase in price, is an insufficient tool for true optimization.[1] This single value represents either the elasticity at a single point on the demand curve (point elasticity) or an average over a specific range (arc elasticity).[1] In most retail scenarios, the demand curve is not a straight line, meaning the elasticity changes at different price levels.[1] A product might be highly elastic at high price points but become inelastic at lower, discounted prices. Therefore, a single elasticity number provides only a localized directional hint (e.g., "we should lower prices") but cannot answer the crucial question: "by how much?"

To find the optimal price, a model must estimate the entire demand function, $q(p)$, which describes the expected quantity (q) that will be sold at any given price (p) within a relevant range.[3] By combining this demand function with the product's marginal cost (

c), one can construct the profit function $\pi(p)=(p-c)\times q(p)$ and then find the price $p*$ that maximizes it.

B2B vs. B2C Pricing Dynamics: The Bid-Response Paradigm

The dynamics of B2B pricing diverge significantly from those in B2C markets, a distinction that has profound implications for modeling. B2C purchases can be impulsive, driven by emotion and marketing. In contrast, B2B purchasing is typically rational, need-based, and often embedded within complex procurement processes, long-term contracts, and negotiations.[4]

A critical insight for B2B modeling is that the primary customer response to price is often not a continuous adjustment of quantity, but a binary decision to accept or reject a quote. A business customer typically knows the quantity they need based on their operational requirements; for example, a manufacturer needs a specific number of components for a production run.[7] A small change in price is unlikely to alter this required quantity. Instead, the price will determine

which supplier wins the deal. This shifts the modeling problem from predicting a continuous quantity to predicting a binary outcome: Win or Loss.[7]

This paradigm shift means that for many B2B scenarios, the relevant model is not a demand curve but a **bid-response function**. This function, which typically assumes a sigmoid (S-shaped) curve, models the probability of winning a deal as a function of the offered price (or price ratio).[8] As the price increases, the win probability decreases. The goal of the model becomes estimating this

Win Probability vs. Price curve, which can then be used to calculate expected profit: Expected Profit(p) = (p - c) * Win Probability(p) * Expected Quantity. This reframing from a regression problem (predicting quantity) to a classification problem (predicting win probability) is a cornerstone of effective B2B price modeling.

The Strategic Role of the Objective Function

While maximizing short-term profit is a common default objective, it is not the only one. Pricing is a powerful strategic lever that can be tuned to achieve different corporate goals.[6] For instance, a company entering a new market might prioritize gaining market share by adopting a

penetration pricing strategy, which involves setting prices lower than the profit-maximizing point to maximize sales volume.[3] Other objectives could include

maximizing total revenue, managing inventory levels for seasonal goods, or maintaining a specific price position relative to competitors.[9]

This strategic flexibility necessitates a crucial architectural decision: the separation of the **elasticity model** from the **optimization module**. The elasticity model's sole job is to estimate the causal response curve (q(p) or prob(p)). The optimization module then takes this curve as an input and finds the optimal price based on a user-selected objective function. For example:

- **Profit Maximization:** max(p - c) * q(p)
- **Revenue Maximization:** max(p * q(p))
- **Volume Maximization:** max q(p)

This separation allows the business to adapt its pricing strategy to changing market conditions or corporate priorities without needing to retrain the core, computationally expensive elasticity model. It transforms the pricing system from a static calculator into a dynamic, strategic tool.

1.2 The Causal Inference Imperative

The most pervasive and critical error in pricing analytics is mistaking correlation for causation. Historical sales data is not a clean record of controlled experiments; it is a messy log of market equilibrium points, where price and quantity are simultaneously determined by a host of underlying factors. A failure to rigorously account for these factors will lead to nonsensical and dangerously misleading elasticity estimates.

Correlation is Not Causation in Pricing

A naive model that simply regresses historical quantity on historical price will almost invariably produce incorrect results. Consider a common scenario: during the peak holiday season, a retailer observes both high sales volumes and high prices for a popular product. A simple correlation-based model would conclude that high prices *cause* high sales—a violation of the fundamental law of demand.[2] The reality, of course, is that a third factor, or

confounder—high seasonal demand—is causing both the price and the quantity to rise. A pricing strategy based on this flawed model would lead to disastrous decisions.

The central task of a robust pricing model is to achieve **causal identification**: to isolate the true, independent effect of a price change on demand, holding all other factors constant.[1]

Identifying Confounding Variables

To isolate the causal effect of price, the model must explicitly account for all relevant confounding variables. In a typical B2B retail context, these include [1]:

- **Seasonality and Temporal Trends:** Holidays, day-of-week effects, and underlying growth or decline in a product's popularity.
- **Promotional Activities:** Discounts, marketing campaigns, and advertising spend that influence demand independently of the base price.
- **Competitive Actions:** Price changes by competitors for substitute products are a primary driver of demand shifts.
- **Product and Customer Attributes:** Changes in the product mix, the introduction of new products, or shifts in the customer base.
- **Macroeconomic Factors:** Broader economic conditions that affect overall purchasing power and business investment.
- **Operational Factors:** Inventory levels (a product cannot be sold if it is out of stock) and even changes to product placement on a website can influence sales.[11]

Advanced machine learning models are particularly well-suited to this task, as they can ingest a wide array of these features and learn the complex, non-linear relationships between them to effectively control for their influence.[14]

The Potential Outcomes Framework

To formalize the concept of causal effects, we turn to the **Potential Outcomes framework**, a cornerstone of modern causal inference.[15] For any given customer-product pair at a specific point in time, we can imagine two potential realities:

1. The outcome (e.g., quantity purchased) if the product were offered at Price A: Y(PA).
2. The outcome if the product were offered at Price B: Y(PB).

The causal effect of changing the price from A to B is the difference between these two potential outcomes: Y(PB)−Y(PA). The fundamental problem of causal inference is that we can only ever observe one of these potential outcomes for any single transaction.[15] We see the quantity purchased at the price that was actually offered; the outcome under the alternative price is unobserved and is known as the

counterfactual.

The goal of a causal inference model is to use the observed data and a set of defensible assumptions to estimate this unobserved counterfactual. Meta-learning algorithms, which will be discussed in detail in Part 3, are explicitly designed to solve this problem by training models to predict outcomes under different treatment (price) scenarios.[18]

The Trade-off of Experimentation

The most reliable way to measure causal effects is through **Randomized Controlled Trials (RCTs)**, commonly known as A/B tests in the business world.[16] By randomly assigning different prices to similar groups of customers, RCTs eliminate confounding variables by design, providing a clean measure of price elasticity.

However, large-scale, continuous experimentation is often impractical. It can be expensive to implement, slow to yield results, and carries the risk of revenue loss or customer dissatisfaction if suboptimal prices are tested.[20] Furthermore, for a business with a vast product catalog and numerous customer segments, the number of required experiments becomes combinatorially explosive.

Therefore, while targeted A/B tests are invaluable for validating models (as discussed in Part 5), the ability to draw robust causal inferences from **observational (historical) data** is a critical capability. Advanced causal inference techniques, powered by machine learning, provide a powerful and scalable way to estimate price elasticities, serving as a vital alternative or complement when extensive live experimentation is not feasible.[22]

Part 2: The Data and Feature Engineering Backbone

The performance of any price optimization model is fundamentally constrained by the quality and richness of the data it is trained on. Building a comprehensive, clean, and feature-rich analytical dataset is the most critical and often most challenging phase of the project. This section provides a detailed blueprint for constructing this data backbone, covering the essential data sources, preprocessing steps, and a "cookbook" of advanced feature engineering techniques specifically designed to capture the complex drivers of B2B pricing.

2.1 Constructing the Analytical Dataset

A robust pricing model requires the integration of data from multiple enterprise systems to create a holistic view of each transaction. The goal is to assemble a single analytical base table where each row represents a pricing event (e.g., a quote or a sales order line) and the columns contain all the relevant features and outcomes.

Core Data Sources

The following data sources are essential for building a powerful B2B pricing model:

- **Transactional Data (ERP/CRM):** This is the heart of the dataset. It must be at the most granular level possible, ideally the quote or sales order line item. Key fields include customer_id, product_sku, timestamp, quoted_price, final_net_price (after all discounts and rebates), quantity_ordered, and, crucially for B2B, a win_loss_flag indicating whether a quote was converted into a sale.[8] Historical win/loss data is the primary source for training bid-response models.[8]
- **Customer Data (CRM):** To understand who is buying, firmographic data is indispensable. This includes static attributes like industry (e.g., manufacturing, healthcare), company_size (by revenue or employee count), geographic_region, and dynamic attributes like customer_since_date or a calculated

customer_loyalty_score.[4] This data is the foundation for customer segmentation.

- **Product Data (PIM/ERP):** A detailed product catalog is needed to understand what is being sold. This includes SKU, product_hierarchy (e.g., category, sub-category, brand), cost_of_goods_sold (COGS) for profit calculation, and detailed product attributes (e.g., specifications, text descriptions, images).[13]
- **Competitor Data (Web Scraping/Third-Party Providers):** Competitor pricing is a powerful external confounder and a direct driver of customer choice.[9] While challenging to obtain reliably, especially for net B2B prices, even list price data from competitor websites or data aggregators can provide a strong signal for the model.[28]
- **Contextual Data (Marketing/Finance Systems):** To control for other factors influencing demand, the model needs data on promotion_schedules, marketing_campaign_timelines, holiday_calendars, and relevant macroeconomic_indicators.[1]

Data Quality and Preprocessing

Raw data from these disparate systems is rarely ready for modeling. A rigorous preprocessing pipeline is required to ensure data integrity.

- **Handling Missing Values:** Missing data is a common problem. For instance, cost_of_goods_sold or customer_firmographics may be incomplete. Simple imputation methods like filling with the mean or median can be a starting point, but more sophisticated techniques like K-Nearest Neighbors (KNN) imputation or model-based imputation may be necessary for critical fields.[29]
- **Outlier Detection and Treatment:** Anomalous transactions (e.g., a price of $0.01, or an unusually large order quantity) can skew model training and should be investigated, flagged, and potentially excluded or capped.
- **Data Consistency:** It is vital to ensure that identifiers like customer_id and product_sku are standardized across all source systems to allow for correct joins.
- **Addressing Low Price Variation:** A significant challenge, particularly in B2B, is that prices for a given product may not change frequently, making it difficult for traditional econometric models to estimate elasticity.[14] Machine learning models, especially deep neural networks, can partially mitigate this. By learning from a rich set of product and customer characteristics, they can infer the price sensitivity of a new product by comparing its attributes to other products with more historical price variation, effectively "borrowing" information across the

catalog.[14]

The following table provides a comprehensive checklist for the data engineering team, outlining the necessary data, its source, and its purpose in the model.

Table 1: Comprehensive Data Requirements for B2B Price Elasticity Modeling

Data Source System	Data Type	Specific Fields	Relevance to Model	Example Reference
ERP/CRM	Transactional	quote_id, customer_id, product_sku, timestamp, quoted_price, net_price, quantity, win_loss_flag	Core data for model training. win_loss_flag is the target for bid-response models. net_price is the treatment variable. quantity is the target for demand models.	[8]
ERP	Product Cost	product_sku, cost_of_goods_sold (COGS)	Essential for calculating profit (p-c) and setting profit-maximization objective.	[3]
CRM	Customer Attributes	customer_id, industry, company_size, region, customer_since_date, loyalty_tier	Features for customer segmentation and confounder control. Critical for hierarchical and individual-level models.	[4]
PIM/ERP	Product Attributes	product_sku, category, brand, technical_specif	Features for product segmentation	[14]

		ications, text_description, image_url	and similarity. Text/image data can be used to create embeddings.	
Web Scraping / 3rd Party	Competitor Prices	timestamp, competitor_name, product_sku, competitor_price	Critical confounder. Directly influences customer willingness-to-pay and win probability.	[9]
Marketing Systems	Contextual	date, promotion_active_flag, marketing_campaign_name, ad_spend	Confounders that must be controlled for to isolate the price effect.	[11]
Internal/External	Temporal	date, holiday_flag, day_of_week, month	Features to capture seasonality and temporal patterns in demand.	[1]

2.2 Advanced Feature Engineering for Pricing Models

Raw data is just the starting point. The process of **feature engineering**—creating new, informative variables from the existing data—is what unlocks the predictive power of a machine learning model. A rich feature set allows the model to understand the nuanced context of each pricing decision.

Price Dynamics Features

These features capture the pricing context of a product relative to its peers and its own history.

- price_ratio_to_category_avg: The product's net price divided by the average net price of all products in its category over the last 30 days. A value > 1 means it's priced at a premium.
- discount_depth: Calculated as (list_price - net_price) / list_price. This measures the level of discounting applied to a transaction.
- price_volatility: The standard deviation of a product's net price over a defined trailing window (e.g., 90 days). High volatility might signal an unstable market or a product in a clearance phase.
- days_since_last_price_change: Captures how long the current price has been in effect.

Customer Value and Behavior Features

These features segment customers based on their relationship with the company, drawing from RFM (Recency, Frequency, Monetary) analysis principles.

- customer_lifetime_value (CLV): A predictive score of the total future profit from a customer.
- avg_order_size and purchase_frequency: Historical measures of customer purchasing habits.
- customer_tenure: The length of time the customer has been doing business with the company (current_date - customer_since_date).
- is_high_value_customer: A binary flag, often based on being in the top quintile of historical spending.

Product-Level Features

These features describe the product's role and performance within the catalog.

- product_sales_velocity: Units sold per period. This can be used to categorize products as 'fast-movers', 'medium-movers', or 'slow-movers'.

- product_lifecycle_stage: A categorical feature indicating if a product is 'New Introduction', 'Growth', 'Mature', or 'End-of-Life'. Elasticity changes dramatically through these stages.[1]
- **Product Embeddings:** For catalogs with rich text descriptions or images, deep learning techniques like Word2Vec or pre-trained vision transformers can be used to create dense numerical vectors (embeddings) for each product.[14] These embeddings capture latent semantic similarities between products that go beyond simple category labels. A model can learn that two differently categorized products are actually close substitutes if their embeddings are similar, a powerful capability for overcoming data sparsity.[14]

Cross-Product Relationship Features

Understanding how products relate to each other is crucial for preventing cannibalization and identifying cross-sell opportunities.

- **Cross-Price Elasticity:** This measures how the demand for Product A changes in response to a price change in Product B.
  - **Substitutes:** If a price increase for Product A leads to an increase in sales of Product B, they are substitutes (e.g., two competing brands of shampoo). Cross-price elasticity is positive.[1]
  - **Complements:** If a price increase for Product A leads to a decrease in sales of Product B, they are complements (e.g., an eBook reader and eBooks). Cross-price elasticity is negative.[1]
- These relationships can be discovered by analyzing co-purchase patterns in historical transaction data and incorporated as features (e.g., avg_price_of_top_substitute).

Hierarchical Features

These features are essential for the hierarchical models discussed in Part 3. For any given SKU, features are created based on the aggregate behavior of its parent groups in the product hierarchy.

- avg_category_margin: The average profit margin for the SKU's product category.

- category_sales_trend: The percentage change in sales for the category over the last quarter.
- brand_price_perception: The average price positioning of the SKU's brand relative to the market.

By building such a rich feature set, the model is equipped to move beyond a simple price-quantity relationship and learn the complex, context-dependent nature of price elasticity in the B2B landscape. This data and feature engineering backbone is the non-negotiable prerequisite for the comparative modeling approaches that follow.

Part 3: Comparative Analysis of Elasticity Modeling Approaches

With a robust data foundation in place, the central task becomes selecting and implementing a modeling strategy to estimate price elasticity. There is no single "best" approach; the optimal choice depends on a trade-off between personalization, data availability, computational cost, and interpretability. This section provides a detailed comparative analysis of two distinct and powerful methodologies: Segment-Level Elasticity, which prioritizes stability and robustness in the face of sparse data, and Individual-Level Elasticity, which aims for the highest degree of personalization by leveraging cutting-edge causal machine learning.

3.1 Approach A: Segment-Level Elasticity Modeling

The primary motivation for segment-level modeling is to address the pervasive challenge of **data sparsity**.[1] In a large B2B retail catalog, it is common for many specific customer-product combinations to have very few, if any, historical transactions with meaningful price variation. Attempting to build a unique model for each such combination would result in unstable, unreliable estimates (a phenomenon known as overfitting). The segment-level approach overcomes this by grouping similar entities together, creating aggregates with sufficient data to build stable and reliable models.[26]

Conceptual Framework and Segmentation Strategy

The core assumption of this approach is that customers or products within a well-defined segment exhibit similar price sensitivity.[25] The goal is to partition the market into a set of homogeneous groups and estimate a single, representative elasticity for each. This is a form of third-degree price discrimination, where different prices are offered to different groups.[6]

The definition of these segments is a critical strategic step, combining business logic with data-driven techniques:

- **Customer Segments:** Customers can be grouped based on observable firmographic or behavioral attributes. Common segmentation variables include industry, company size, geographic region, customer tenure, or a composite loyalty score.[4] For example, a business might hypothesize that large enterprise customers are less price-sensitive than small businesses.
- **Product Segments:** Products can be grouped based on the existing product hierarchy (e.g., all products within the 'industrial lubricants' sub-category), price band (e.g., premium, mid-tier, economy), or sales velocity (e.g., fast-moving vs. slow-moving items).[24]
- **Combined Segments:** The most powerful segmentation often occurs at the intersection of customer and product attributes (e.g., "small businesses in the Midwest buying premium-tier products").

Modeling Technique: Hierarchical Models

Once segments are defined, one could naively build a separate model for each segment. However, this approach still struggles with segments that have limited data and fails to leverage similarities across segments. A far superior technique is the use of **Hierarchical Models**, also known as Mixed-Effects Models.[11]

A hierarchical model estimates a separate price elasticity coefficient for each micro-segment, but with a crucial twist: it simultaneously models these coefficients as being drawn from a common distribution defined at a higher level of the hierarchy (e.g., the product category or customer industry level).[11] This structure allows segments with sparse data to "borrow statistical strength" from more data-rich

segments within the same parent group.

For example, when estimating the elasticity for a specific, low-volume SKU, the model will be informed not only by the few data points for that SKU but also by the average elasticity of all other SKUs in its category. This leads to more stable and reasonable estimates, a phenomenon known as "shrinkage," where extreme estimates from sparse data are pulled toward the group mean. The **Bayesian Hierarchical Linear Regression** is a particularly powerful implementation of this concept, as it naturally handles uncertainty and allows for the incorporation of prior business knowledge.[26] This approach effectively balances the need for segment-specific estimates with the reality of data limitations, providing a robust solution that avoids the pitfalls of either modeling all segments together (which is too general) or modeling each one separately (which overfits).[26]

Strengths and Limitations

- **Strengths:** The primary advantage is its **robustness to data sparsity**, making it a highly practical starting point for most B2B businesses. The models are generally less computationally expensive to train and maintain than individual-level models. Furthermore, the segment-level results are often more **interpretable** and align well with how business managers traditionally think about the market, facilitating adoption.
- **Limitations:** The key drawback is that it provides an *average* elasticity for an entire segment. It cannot capture the heterogeneity *within* a segment. For example, while the "small business" segment may be price-sensitive on average, some individual small businesses within it might be highly loyal and insensitive to price. This approach misses the opportunity for more granular, personalized pricing, which is the domain of the individual-level models.

3.2 Approach B: Individual-Level Elasticity Modeling (Causal ML)

This approach represents the frontier of pricing science, aiming to achieve what economists call "perfect price discrimination" or first-degree price discrimination: a unique, optimized price for every individual customer-product interaction.[6] The goal is

to estimate the

Conditional Average Treatment Effect (CATE), which is the causal effect of a price change for a *specific* unit (e.g., a customer), conditional on their unique set of features X.

Conceptual Framework: Causal Meta-Learners

Estimating CATE from observational data is a complex causal inference problem. **Meta-learners** are a class of algorithms designed specifically for this task. They are "meta" because they use standard, off-the-shelf machine learning models (e.g., Gradient Boosting, Random Forests, Neural Networks), referred to as "base learners," as building blocks within a larger causal estimation framework.[34] This allows the data scientist to leverage the predictive power of complex ML models while adhering to a sound causal structure. We will focus on three prominent meta-learners.

- **S-Learner (Single-Learner):** This is the most straightforward meta-learner. A single ML model is trained to predict the outcome Y (e.g., quantity or win/loss) using both the customer/product features X and the treatment variable (price P) as inputs: $Y=M(X,P)$. To estimate the CATE for an individual, one simply queries the model twice: once with the price set to a treatment value ($P=p1$) and once with the price set to a control value ($P=p0$). The CATE is the difference: $\hat{\tau}(X)=M(X,p1)-M(X,p0)$.[18] The primary weakness of the S-learner is that if the price is not a very strong predictor of the outcome compared to other features, the regularization inherent in many ML models may cause it to down-weight or even ignore the price feature, biasing the estimated effect towards zero.[19]
- **T-Learner (Two-Learner):** To address the S-learner's main weakness, the T-learner builds two separate models.[15] One model, $M0$, is trained exclusively on the data from the control group (e.g., transactions at a low price point). The second model, $M1$, is trained exclusively on data from the treated group (e.g., transactions at a high price point). The CATE for an individual with features X is then estimated as the difference in the predictions from these two models: $\hat{\tau}(X)=M1(X)-M0(X)$.[15] This approach forces the models to learn the price effect but can be problematic if one of the treatment groups is significantly smaller than the other, as the model for the smaller group may be poorly estimated.[19]
- **X-Learner (Cross-Learner):** The X-learner, proposed by Künzel et al. (2019), is a

state-of-the-art meta-learner designed to be robust, particularly in common scenarios where treatment groups are imbalanced (e.g., far more sales at the standard price than at a promotional price).[19] It is a multi-stage process that cleverly uses the full dataset to estimate the CATE [15]:

1. **Stage 1 (Nuisance Models):** Train two outcome models, M0 and M1, exactly as in the T-learner.
2. **Stage 2 (Imputed Treatment Effects):** For each individual in the **treated** group, impute their counterfactual outcome using the control model: $\hat{Y}_i(0) = M_0(X_i)$. Then, calculate their imputed treatment effect as their actual outcome minus the imputed one: $D_i^1 = Y_i - \hat{Y}_i(0)$. Similarly, for each individual in the **control** group, impute their counterfactual outcome using the treated model: $\hat{Y}_i(1) = M_1(X_i)$, and calculate their imputed treatment effect: $D_i^0 = \hat{Y}_i(1) - Y_i$.
3. **Stage 3 (Effect Models):** Train two new ML models. Model τ_1 is trained to predict the imputed effects D^1 using the features X from the treated group. Model τ_0 is trained to predict the imputed effects D^0 using the features X from the control group.
4. **Stage 4 (Final CATE):** The final CATE for any individual is a weighted average of the predictions from the two effect models: $\hat{\tau}(X) = g(X) \cdot \tau_0(X) + (1 - g(X)) \cdot \tau_1(X)$. The weighting function, $g(X)$, is a **propensity score model**—a separate model trained to predict the probability of an individual receiving the treatment, given their features X. This weighting scheme gives more influence to the effect estimate that was trained on the larger group, enhancing robustness.

Practical Implementation with EconML

Implementing these meta-learners from scratch is complex. Fortunately, specialized Python libraries like Microsoft's **EconML** provide robust, production-ready implementations.[37] A conceptual implementation of an X-Learner using EconML would look as follows:

Python

```
# Conceptual Python Code using EconML
from econml.metalearners import XLearner
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LogisticRegression

# 1. Define the base learners for outcome and effect models
models = RandomForestRegressor(n_estimators=100, min_samples_leaf=10)

# 2. Define the propensity score model
propensity_model = LogisticRegression()

# 3. Instantiate the X-Learner
# It takes the outcome models, effect models (can be same), and propensity model
x_learner = XLearner(models=models, propensity_model=propensity_model)

# 4. Fit the model
# Y: outcome (e.g., quantity), T: treatment (e.g., price or binary high/low price flag)
# X: features, W: confounders (optional, can be part of X)
x_learner.fit(Y, T, X=X_features)

# 5. Estimate the CATE for new customers
# X_test contains the features of customers for whom we want to estimate elasticity
individual_elasticities = x_learner.effect(X_test)
```

This library abstracts away the complex multi-stage fitting process, allowing the data science team to focus on feature engineering and model selection.[36]

Strengths and Limitations

- **Strengths:** This approach offers the highest potential for profit optimization by enabling true one-to-one pricing. It directly models the heterogeneous effects that are averaged out in segment-level approaches, capturing the nuances of individual customer behavior.
- **Limitations:** The primary drawback is its **high data requirement**. It performs poorly in sparse data settings. The models are **computationally expensive** to train and score. Finally, their complexity can make them a "black box," posing a significant challenge for **interpretability and business adoption** without the use of supplementary tools like SHAP (discussed in Part 5).[4]

Comparative Summary

The choice between these two approaches is a strategic one, balancing precision against practicality. The following table provides a direct comparison to guide this decision.

Table 2: Comparative Analysis of Modeling Approaches

Dimension	Segment-Level (Hierarchical Model)	Individual-Level (Causal ML / X-Learner)
Key Assumption	Entities within a segment share a common price elasticity.	Price elasticity is unique to each individual and can be estimated from their features.
Data Sparsity Tolerance	**High**. Explicitly designed to handle sparse data by "borrowing strength" across segments.	**Low**. Requires sufficient data at the individual feature level to learn heterogeneous effects.
Computational Cost	**Moderate**. Fewer models to train, though Bayesian methods can be intensive.	**High**. Involves training multiple complex ML models in several stages.
Personalization Potential	**Medium**. Prices are optimized for segments (e.g., all small businesses), not individuals.	**High**. Aims for a unique, optimal price for every customer-product interaction.
Interpretability	**Higher**. The concept of segment-level elasticity is more intuitive for business stakeholders.	**Lower**. The model is a complex "black box" requiring specialized tools like SHAP to explain its reasoning.
Primary B2B Use Case	Broad-based pricing for the entire catalog, especially for products with limited history or in markets with less data maturity.	Strategic pricing for high-value customers or key product categories where the potential profit lift justifies the complexity.

Key References	11	18

Ultimately, a hybrid strategy is often the most effective. A B2B organization could begin by implementing a robust, segment-level hierarchical model across its entire product catalog to establish a solid baseline. Then, it can strategically deploy individual-level X-Learner models on the most critical, data-rich, and high-value segments of the business to capture additional profit lift where it matters most.

Part 4: Determining Optimal Stretch and Floor Prices

The output of an elasticity model—whether it's a demand curve, a bid-response function, or a set of individual treatment effects—is not the end goal. It is a critical input into the final stage of the process: calculating the specific, actionable price points that drive business objectives. This section details the methodologies for translating model outputs into the two key prices requested: the profit-maximizing 'stretch price' and the data-driven 'floor price'. It will cover the mathematical optimization required for the stretch price and explore several advanced machine learning techniques for estimating a floor price that moves beyond simple cost-plus rules.

4.1 From Elasticity to Profit: Optimizing the Stretch Price

The 'stretch price' is defined as the price that maximizes total profit. Once the elasticity model from Part 3 provides a reliable function for the expected demand q(p) or the win probability prob(p) for a given price p, we can formally define the expected profit function.

The Profit Maximization Formula

For a given customer-product interaction, the expected profit π(p) at a given price p

is a function of the price, the marginal cost c (typically the Cost of Goods Sold), and the expected volume.

- If using a demand curve model, the formula is:

  $\pi(p) = (p - c) \times q(p)$
- If using a bid-response model, where the model predicts the probability of winning a deal of a known quantity Qdeal, the formula is:

  $\pi(p) = (p - c) \times prob(p) \times Qdeal$

The task is to find the price $p*$ that maximizes this function $\pi(p)$.[3]

Numerical Optimization

For simple, linear demand curves, this optimal price can sometimes be solved analytically by taking the derivative of the profit function, setting it to zero, and solving for p. However, the demand and probability functions learned by complex machine learning models (like Random Forests or Neural Networks) do not have a simple, closed-form equation. Therefore, we must rely on **numerical optimization** algorithms to find the maximum.

These algorithms work by iteratively testing different prices and intelligently moving in the direction that increases the profit. A common and powerful tool for this in Python is the scipy.optimize.minimize function.[41] A key detail is that this function, as its name suggests, is designed for minimization. To find the price that

maximizes profit, we simply ask the function to *minimize* the **negative** of the profit function.[44]

Implementation with SciPy and Business Constraints

A conceptual implementation involves defining a Python function that calculates the negative profit for a given price and then passing it to the optimizer.

Python

```python
# Conceptual Python Code for Profit Maximization
from scipy.optimize import minimize
import numpy as np

# Assume 'elasticity_model' is our trained model (e.g., an X-Learner)
# and 'features' are the attributes for a specific customer-product pair.
# 'cost' is the marginal cost of the product.

def negative_profit(price, elasticity_model, features, cost):
    """
    Calculates the negative of the expected profit for a given price.
    The optimizer will try to make this value as small as possible.
    """
    price = price # Optimizer passes price as an array

    # Get the predicted demand (or win probability * quantity) from the model
    predicted_demand = elasticity_model.predict(features, price)

    # Calculate profit
    profit = (price - cost) * predicted_demand

    # Return the negative of the profit
    return -profit

# --- Optimization ---
# Initial guess for the price
initial_price_guess = np.array([100.0])

# Define bounds for the price search. Price cannot be below cost.
# It also cannot exceed a reasonable upper limit (e.g., 3x list price).
price_bounds = [(cost, cost * 5)]

# Run the optimization
result = minimize(
    fun=negative_profit,
    x0=initial_price_guess,
    args=(elasticity_model, features, cost),
```

```
    method='SLSQP', # A method that supports bounds and constraints
    bounds=price_bounds
)

# The optimal stretch price is in the result object
stretch_price = result.x
max_profit = -result.fun
```

This optimization framework is also highly flexible and can incorporate various **business rules as constraints**.[9] For example, a business might have rules like "the final price cannot be more than 5% above the closest competitor's price" or "the profit margin must be at least 20%." These rules can be formulated as inequality constraints and passed to the

minimize function, ensuring that the recommended stretch price is not only mathematically optimal but also commercially viable and strategically aligned.

4.2 Machine Learning for the Floor Price

The 'floor price' is the minimum acceptable price for a deal. Traditionally, this is often set using a simple "cost-plus" logic (e.g., cost * 1.2). However, a data-driven approach can yield a more strategic floor price that reflects market conditions and customer willingness to pay. This section explores three distinct ML-based methods for estimating a floor price.

Method 1: Modeling Win-Loss Probability (Bid-Response)

This is the most causally sound approach for B2B environments. As established in Part 1, the bid-response function models the probability of winning a deal at a given price.[8]

- **Concept:** The floor price is not a direct output of the model but a strategic decision based on its predictions. The business must define its risk tolerance by setting a minimum acceptable win probability.
- **Model:** A probabilistic classification model (e.g., Logistic Regression, XGBoost Classifier) is trained on historical quote data. The target variable is the binary

win_loss_flag. The features include the price_ratio (net price / list price), customer attributes, product attributes, and competitive information.[8]

- **Setting the Floor:** For a new quote, the model can simulate the win probability across a range of potential prices. The business can then establish a rule, for example: "The floor price is the lowest price we can offer while maintaining at least a 30% probability of winning the deal." This 30% threshold becomes a tunable business parameter, allowing management to become more aggressive (lower the threshold) or more conservative (raise the threshold) based on strategic goals.

Method 2: Predicting the Reservation Price Lower Bound

This is a more advanced and experimental approach that attempts to directly estimate the customer's reservation price (their maximum willingness to pay).[47] Since the true reservation price is unobservable, the model instead predicts a plausible lower bound for it.

- **Concept:** This method adapts techniques from other domains, such as programmatic advertising, where models are built to predict the optimal reserve price in an auction.[48] Instead of predicting a single point value for the likely winning bid, the model predicts a confidence interval.
- **Model:** A model is trained to predict a specific low quantile of the distribution of winning prices for similar historical deals. This can be achieved using **Quantile Regression** or by designing a custom neural network with a specialized **quality-driven (QD) loss function** that penalizes the model based on the width and coverage of its predicted interval.[48]
- **Setting the Floor:** The predicted lower bound (e.g., the 10th percentile of the likely winning bid distribution) can be used directly as an aggressive, data-driven floor price. This tells the salesperson the absolute minimum they should consider, based on historical evidence of what it takes to win similar deals.

Method 3: A Regression Approach on Accepted Prices

This is a simpler, more direct method that frames the problem as a standard

regression task.

- **Concept:** The model learns to predict the final accepted price for a deal based on its characteristics.
- **Model:** Using only the dataset of **"won" deals**, a regression model (e.g., XGBoost, Random Forest) is trained to predict the net_price based on all available customer, product, and deal context features.[29]
- **Setting the Floor:** The floor price can be derived from the model's prediction. For example, the floor could be set as predicted_price * 0.95 or predicted_price - 1 * standard_deviation_of_error. This approach is less causally rigorous than the bid-response method because it suffers from selection bias (it only learns from winning bids), but it can serve as a highly practical and often effective baseline.

4.3 Advanced Modeling Technique: Stacking Regressors for Price Prediction

To enhance the accuracy of the regression-based models used in Method 3 (for floor price) or even in the base learners of the meta-learners (for stretch price), an advanced ensemble technique called **stacking** can be employed.

- **Concept:** Stacking, or stacked generalization, combines the predictions of multiple different machine learning models to produce a final prediction that is often more accurate than any of the individual models.[51] It operates in two levels:
  1. **Level-0 (Base Models):** A diverse set of models (e.g., a linear model, a tree-based model, a boosting model) are trained on the original training data.
  2. **Level-1 (Meta-Model):** The predictions from the Level-0 models are then used as input features to train a final "meta-model." This meta-model learns the optimal way to combine the base predictions, effectively learning to trust certain models more in different situations.[51]
- **Implementation:** The scikit-learn library in Python provides a convenient StackingRegressor class for this purpose.[54]
  - **Base Models (Level-0):** The key to successful stacking is model diversity. A good choice of base models would be RidgeCV (a robust linear model), RandomForestRegressor (a bagging ensemble), and XGBoost (a gradient boosting ensemble).[55] Each captures different types of patterns in the data.
  - **Meta-Model (Level-1):** The meta-model is often a simple, stable model like LassoCV or RidgeCV. Its job is not to learn complex patterns from the original features, but to learn the best linear combination of the base model

predictions.[54]

By using a stacking regressor, the predictive accuracy for the floor price (Method 3) or the nuisance components of the X-learner can be significantly improved, leading to more reliable and profitable final price recommendations.

Part 5: Validation, Interpretation, and Operationalization

Developing a technically sophisticated pricing model is only half the battle. For a pricing optimization initiative to succeed, the model must be rigorously validated, its recommendations must be trusted by the business users who depend on them, and it must be seamlessly integrated into the organization's daily workflows. This final section provides a comprehensive framework for achieving these goals, covering model validation through offline and online testing, unlocking the "black box" of complex models using interpretability tools, and outlining a practical roadmap for operationalizing the system across people, processes, and technology.

5.1 A Framework for Model Validation and A/B Testing

A multi-layered validation strategy is essential to ensure that the model is not only statistically sound but also drives real business value. This involves both offline evaluation on historical data and online evaluation in a live market environment.

Offline Validation

Before deploying any model, it must be thoroughly tested on historical data that it was not trained on.

- **Standard Holdout Validation:** The most fundamental step is to split the historical data into a training set and a holdout (or test) set. The model is trained on the former and evaluated on the latter. For regression tasks (e.g., predicting demand or a floor price), common metrics include Root Mean Squared Error

(RMSE) and Mean Absolute Error (MAE).[29] For classification tasks (e.g., predicting win/loss), metrics like Area Under the ROC Curve (AUC), Precision, and Recall are used.[8]

- **Causal Validation:** For causal models like the X-learner, standard predictive metrics are insufficient. We need to validate the causal estimate itself. Specialized techniques are required:
  - **Refutation Tests:** Libraries like DoWhy in Python offer refutation methods that test the robustness of the causal estimate.[38] For example, one can add a random common cause to the data and check if the estimated effect changes significantly (it shouldn't). Another test involves replacing the treatment variable (price) with a random placebo variable; the model should correctly estimate a near-zero effect.
  - **Subgroup Validation:** The model's predictions should align with business intuition. For example, one might expect a higher price elasticity for a customer segment known to be price-sensitive. Validating that the CATE estimates differ in expected ways across key business segments can build confidence.

Online Validation (A/B Testing)

The ultimate arbiter of a pricing model's value is its performance in the real world. A carefully designed **field experiment**, or A/B test, provides the definitive proof of its impact.[2]

- **Experimental Design:** A group of comparable customer-product pairs should be selected for the test. This group is then randomly split:
  - **Control Group (Group A):** Continues to receive prices determined by the existing pricing strategy (e.g., manual pricing, cost-plus).
  - **Treatment Group (Group B):** Receives prices recommended by the new machine learning model.
- **Measurement and Metrics:** The experiment is run for a predetermined period (e.g., one fiscal quarter). At the end of the period, key business metrics are compared between the two groups. The primary success metric is typically **total profit**. Other important metrics to monitor include total revenue, sales volume, win rate, and customer churn.[2] A successful model should demonstrate a statistically significant increase in profit for the treatment group without causing an unacceptable decline in other key metrics. Results from such tests, like a

specialty retailer achieving a 22% net revenue lift in test stores, provide powerful justification for a full-scale rollout.[2]

5.2 Unlocking the Black Box: Model Interpretability with SHAP

One of the greatest barriers to the adoption of AI-driven pricing is a lack of trust from the sales and merchandising teams who are ultimately responsible for revenue.[4] Complex models like X-learners or stacking regressors are often perceived as "black boxes," making it impossible for users to understand the rationale behind a given price recommendation. This is where model interpretability becomes critical.[39]

Introducing SHAP (SHapley Additive exPlanations)

SHAP is a state-of-the-art, game theory-based framework for explaining the output of any machine learning model.[57] For any single prediction, SHAP assigns each feature a specific value representing its contribution to pushing the prediction away from the baseline (or average) prediction. The sum of these SHAP values equals the difference between the final prediction and the baseline, providing a complete, additive explanation.[57]

Practical Application and Interpretation

SHAP provides both global and local explanations, which are invaluable for building trust and collaborating between data scientists and domain experts.[40]

- **Global Interpretability (Feature Importance):** By aggregating the SHAP values for each feature across the entire dataset, we can create summary plots that show which features are the most influential *on average*. This answers high-level strategic questions like, "What are the top three drivers of price sensitivity in our customer base?".[57]
- **Local Interpretability (Individual Prediction Explanation):** This is the most powerful application for building trust with sales teams. For any single price

recommendation, a SHAP **force plot** can be generated. This plot visually shows which features pushed the price up and which pushed it down for that specific deal. For example, a salesperson could see a recommendation and its explanation: "The recommended price is **higher** than average because this customer is small and has low purchase frequency (low sensitivity), but it is **lower** than it could be because of an aggressive competitor price and the fact that this is a slow-moving product.".[57] This level of transparency transforms the model from an opaque instruction-giver into a trusted advisor.

Conceptual code for generating SHAP explanations using the shap library is straightforward:

Python

```
# Conceptual Python Code for SHAP
import shap
import pandas as pd

# Assume 'model' is our trained pricing model (e.g., X_learner)
# and 'X_data' is the feature set
explainer = shap.Explainer(model, X_data)
shap_values = explainer(X_data)

# --- Global Interpretation ---
# Summary plot showing global feature importance
shap.summary_plot(shap_values, X_data)

# --- Local Interpretation ---
# Force plot explaining the first prediction in the dataset
shap.force_plot(explainer.expected_value, shap_values[0,:], X_data.iloc[0,:])
```

By integrating these SHAP visualizations directly into the pricing dashboard or CPQ tool, the "why" behind each AI-generated price becomes clear, dramatically increasing the likelihood of adoption and effective use.

5.3 From Model to Market: An Implementation Roadmap

Successfully operationalizing an AI-powered pricing system requires a coordinated effort across people, processes, and technology.

People

- **Centralized Pricing Team:** Best-in-class organizations establish a centralized pricing team or Center of Excellence (CoE).[9] This cross-functional team should include data scientists (who build and maintain the models), pricing strategists (who understand the business context and set objectives), and category managers (who have deep domain expertise). This team owns the pricing engine, monitors its performance, and acts as the bridge between the technology and the field sales organization.

Process

- **Dynamic "Read and React" Workflow:** AI-powered pricing enables a shift away from static, annual price reviews towards a dynamic, continuous process. The pricing team should establish a cadence (e.g., weekly or monthly) for monitoring model performance against strategic goals and identifying deviations.[9] This "read and react" capability allows the business to respond swiftly to competitor price moves, cost inflation, or changes in customer behavior.
- **Feedback Loop:** The process must include a clear workflow for the sales team to provide feedback and, when necessary, override model recommendations. These overrides should not be discarded; they are valuable data points. The reason for the override (e.g., "competitor offered a lower price not in the system") should be captured and fed back into the model for future retraining, creating a continuous learning loop.[8]
- **Discounting Policy:** The model's outputs should inform a clear, structured discounting policy. This empowers the sales team with well-defined guardrails, reducing ad-hoc discounting that can erode margins.[8]

Technology

- **Integrated Data Platform:** The foundation of the system is a fully integrated data platform that consolidates all the necessary data sources (from Part 2) into a single source of truth, updated in near real-time.[9]
- **System Integration:** The pricing engine cannot be a standalone tool. Its outputs (the recommended stretch and floor prices) must be seamlessly integrated into the core commercial systems where pricing decisions are made and executed. This includes the Customer Relationship Management (CRM) system, Configure-Price-Quote (CPQ) tools, ERP systems, and any B2B eCommerce platforms.[9] This automation is key to delivering the right price at the right time.
- **User Interface (UI) and Dashboards:** A user-friendly interface is critical for making the model's power accessible. This dashboard should allow pricing managers to run "what-if" scenarios (e.g., "what happens to our total profit if we prioritize market share for the next quarter?"), visualize the elasticity curves, and, most importantly, view the SHAP explanations for individual price recommendations.[9] This UI is the window into the model, transforming the "black box" into a transparent and interactive decision-support tool.

Conclusion and Recommendations

This report has detailed a comprehensive framework for developing and implementing a sophisticated, machine learning-driven price optimization system in a B2B retail context. The analysis underscores that such an initiative is not merely a technical exercise but a profound strategic undertaking that requires a solid foundation in causal economics, robust data engineering, and a deliberate plan for organizational adoption.

The core of the analysis presented two distinct but powerful modeling philosophies: a **Segment-Level approach** using hierarchical models to ensure stability in the face of sparse data, and an **Individual-Level approach** using advanced causal meta-learners like the X-learner to achieve maximum personalization. The optimal path forward is rarely to choose one exclusively over the other.

Therefore, the primary recommendation is to adopt a **phased, hybrid strategy**:

1. **Foundation First (Phase 1):** Begin by implementing a **Segment-Level**

Hierarchical Model across the entire product catalog. This approach is more resilient to the data sparsity common in B2B environments and provides a robust, interpretable baseline. It will deliver immediate value by moving beyond simplistic cost-plus or competitor-matching rules and will build organizational confidence in data-driven pricing. The focus during this phase should be on establishing the critical data and feature engineering backbone, as this is a prerequisite for any advanced modeling.

2. **Strategic Personalization (Phase 2):** Once the foundational model is operational and validated, strategically deploy **Individual-Level Causal Models (X-Learners)** on a targeted basis. These models should be focused on the segments of the business with the highest potential return on investment, characterized by:
    - **High Value:** Key customer accounts or high-margin product categories where even small improvements in pricing can yield significant profit lift.
    - **High Volume/Data Richness:** Segments with sufficient historical transaction data and price variation to reliably train these more complex models.
    - **Strategic Importance:** Product lines central to the company's growth or competitive strategy.

Regardless of the chosen modeling depth, several principles are universal and non-negotiable for success:

- **Embrace a Causal Mindset:** All modeling efforts must be grounded in the principles of causal inference. Actively identifying and controlling for confounding variables is essential to avoid spurious correlations and build models that accurately reflect true market dynamics.
- **Prioritize Interpretability:** The "black box" problem is the single greatest threat to adoption. The integration of interpretability tools like SHAP is not an optional add-on; it is a core requirement for building trust and facilitating collaboration between data science and business teams.
- **Invest in the End-to-End System:** A brilliant model that is not integrated into sales workflows is worthless. Success requires investment in the full operational stack: the people (a centralized pricing team), the processes (a dynamic "read and react" workflow with feedback loops), and the technology (an integrated data platform and a user-friendly interface).

By following this strategic roadmap—building a robust foundation, personalizing where it matters most, and relentlessly focusing on causality and trust—a B2B organization can transform its pricing function from a reactive cost center into a

proactive, data-driven engine for sustainable profit growth.

Works cited

1. Price Elasticity in Retail: Definition & Impact - Quicklizard, accessed July 30, 2025, https://quicklizard.com/blog/what-is-price-elasticity-how-does-it-impact-retail/
2. Using elasticity modeling to test retail pricing | Articles - Quirks Media, accessed July 30, 2025, https://www.quirks.com/articles/using-elasticity-modeling-to-test-retail-pricing
3. How to Get the Price Right. Tools for optimal price setting | by Emily Glassberg Sands | Teconomics | Medium, accessed July 30, 2025, https://medium.com/teconomics-blog/how-to-get-the-price-right-9fda84a33fe5
4. AI and dynamic pricing in B2B industrial companies: Why it's not a match made in heaven., accessed July 30, 2025, https://www.simon-kucher.com/en/insights/ai-and-dynamic-pricing-b2b-industrial-companies-why-its-not-match-made-heaven
5. Understanding Price Elasticity: A Guide for B2B Success - sposea, accessed July 30, 2025, https://www.sposea.com/blog/understanding-b2b-price-elasticity/
6. Top B2B Pricing Strategies: Examples and Guide | Vendavo, accessed July 30, 2025, https://www.vendavo.com/pricing/b2b-pricing-strategy-guide/
7. Yes, You Can Measure Price Elasticity in B2B | podcast - Zilliant, accessed July 30, 2025, https://zilliant.com/podcasts/yes-you-can-measure-price-elasticity-in-b2b
8. Machine Learning for B2B Pricing - Wipro, accessed July 30, 2025, https://www.wipro.com/analytics/machine-learning-for-b2b-pricing/
9. Overcoming Retail Complexity with AI-Powered Pricing | BCG - Boston Consulting Group, accessed July 30, 2025, https://www.bcg.com/publications/2024/overcoming-retail-complexity-with-ai-powered-pricing
10. Price Elasticity of Demand Automation - Mosaic Data Science, accessed July 30, 2025, https://mosaicdatascience.com/2016/08/30/price-elasticity-of-demand-cs/
11. Machine Learning for Retail Pricing to Maximize Revenue from Mosaic, accessed July 30, 2025, https://mosaicdatascience.com/2023/01/23/maximizing-revenue-with-machine-learning-for-retail-pricing/
12. Causal Inference in Machine Learning for Accurate Prediction in Dynamic Pricing Models - ISAR Publisher, accessed July 30, 2025, https://article.isarpublisher.com/download/722
13. NBER WORKING PAPER SERIES ALGORITHMIC PRICING: IMPLICATIONS FOR MARKETING STRATEGY AND REGULATION Martin Spann Marco Bertini Ode, accessed July 30, 2025, https://www.nber.org/system/files/working_papers/w32540/w32540.pdf
14. Neural Network Approach to Demand Estimation and Dynamic Pricing in Retail - arXiv, accessed July 30, 2025, https://arxiv.org/html/2412.00920v1
15. Causal Inference: Intro To Meta Learners | by Mandeep Singh | Medium, accessed July 30, 2025,

https://medium.com/@mandeep0405/causal-inference-intro-to-meta-learners-1e35383a19d2
16. Causal Inference for Data Science - Aleix Ruiz de Villa - Manning Publications, accessed July 30, 2025, https://www.manning.com/books/causal-inference-for-data-science
17. Causal Inference for The Brave and True - Matheus Facure, accessed July 30, 2025, https://matheusfacure.github.io/python-causality-handbook/landing-page.html
18. Causal Prediction Algorithms — Dataiku DSS 14 documentation, accessed July 30, 2025, https://doc.dataiku.com/dss/latest/machine-learning/causal-prediction/causal-prediction-algorithms.html
19. T-learners, S-learners and X-learners | Statistical Odds & Ends - WordPress.com, accessed July 30, 2025, https://statisticaloddsandends.wordpress.com/2022/05/20/t-learners-s-learners-and-x-learners/
20. Pricing Experimental Design: Causal Effect, Expected Revenue and Tail Risk, accessed July 30, 2025, https://proceedings.mlr.press/v202/simchi-levi23a.html
21. Pricing Experimental Design: Causal Effect, Expected Revenue and ..., accessed July 30, 2025, https://pubsonline.informs.org/doi/10.1287/mnsc.2023.03209
22. [2303.16660] Price Optimization Combining Conjoint Data and Purchase History: A Causal Modeling Approach - arXiv, accessed July 30, 2025, https://arxiv.org/abs/2303.16660
23. Price Optimization Combining Conjoint Data and Purchase History: A Causal Modeling Approach - Project MUSE, accessed July 30, 2025, https://muse.jhu.edu/article/929116
24. A Complete Guide to Implementing Price Elasticity Using a Smart ..., accessed July 30, 2025, https://www.symson.com/blog/implementing-price-elasticity-in-smart-pricing-tool
25. Segment-Based Pricing Strategy | Examples & Benefits - Symson, accessed July 30, 2025, https://www.symson.com/pricing-strategies/all-about-segment-based-pricing-strategy
26. #TBT: What Works When Traditional Price Elasticity Fails | blog, accessed July 30, 2025, https://zilliant.com/blog/tbt-what-works-when-traditional-price-elasticity-fails
27. How Machine Learning is reshaping Price Optimization | Tryolabs, accessed July 30, 2025, https://tryolabs.com/blog/price-optimization-machine-learning
28. Optimizing Prices with Machine Learning (ML) | Priceva, accessed July 30, 2025, https://priceva.com/blog/optimizing-prices-with-machine-learning
29. House Price Prediction using Machine Learning Models - DataHen, accessed July 30, 2025, https://www.datahen.com/blog/house-price-prediction-using-machine-learning-models/

30. House Price Prediction using Machine Learning in Python - GeeksforGeeks, accessed July 30, 2025, https://www.geeksforgeeks.org/machine-learning/house-price-prediction-using-machine-learning-in-python/
31. Neural Network Approach to Demand Estimation and Dynamic ..., accessed July 30, 2025, https://arxiv.org/abs/2412.00920
32. Elasticity vs Inelasticity of Demand: 5 Main Differences that Brands Should Know - Symson, accessed July 30, 2025, https://www.symson.com/blog/elasticity-vs-inelasticity-of-demand
33. Price Elasticity Estimation by Causal-inference Methods: Lessons from Real-world Applications - POLITesi, accessed July 30, 2025, https://www.politesi.polimi.it/retrieve/988af470-f745-4fae-b4c8-3f6596e32258/2022_07_Mucollari_Minotti_02.pdf
34. Methodology - causalml documentation - Read the Docs, accessed July 30, 2025, https://causalml.readthedocs.io/en/latest/methodology.html
35. 21 - Meta Learners — Causal Inference for the Brave and True - Matheus Facure, accessed July 30, 2025, https://matheusfacure.github.io/python-causality-handbook/21-Meta-Learners.html
36. Heterogeneous treatment effect and Meta Learners - Towards Data Science, accessed July 30, 2025, https://towardsdatascience.com/heterogeneous-treatment-effect-and-meta-learners-38fbc3ecc9d3/
37. py-why/EconML: ALICE (Automated Learning and Intelligence for Causation and Economics) is a Microsoft Research project aimed at applying Artificial Intelligence concepts to economic decision making. One of its goals is to build a toolkit that combines state-of-the-art machine learning techniques with econometrics in order to bring automation - GitHub, accessed July 30, 2025, https://github.com/py-why/EconML
38. Meta-Learners — econml 0.16.0 documentation, accessed July 30, 2025, https://econml.azurewebsites.net/spec/estimation/metalearners.html
39. Pricing optimization in retail: Maximizing profits with AI | by COAX Team - Medium, accessed July 30, 2025, https://medium.com/coax-software-blog/pricing-optimization-in-retail-maximizing-profits-with-ai-770efab44f49
40. Case - Interpretable Price Elasticity Estimation, accessed July 30, 2025, https://www.interpretable.ai/solutions/pricing-elasticity/
41. Mastering SciPy Optimize for OR - Number Analytics, accessed July 30, 2025, https://www.numberanalytics.com/blog/ultimate-guide-scipy-optimize-operations-research
42. Optimization (scipy.optimize) — SciPy v1.16.1 Manual - Numpy and Scipy Documentation, accessed July 30, 2025, https://docs.scipy.org/doc/scipy/tutorial/optimize.html
43. minimize — SciPy v1.16.1 Manual, accessed July 30, 2025, https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.htm

l
44. Maximize Profit with Scipy Optimize - Nikhil Rao's Blog, accessed July 30, 2025, https://nikhilrao.blog/maximize-profit-with-scipy-optimize
45. Minimization/maximization of functions - The Kitchin Research Group, accessed July 30, 2025, https://kitchingroup.cheme.cmu.edu/f19-06623/10-min-max.html
46. What is Profit Optimization? - Vendavo, accessed July 30, 2025, https://www.vendavo.com/glossary/what-is-profit-optimization/
47. Key Insight on Reservation Price for IO Firms - Number Analytics, accessed July 30, 2025, https://www.numberanalytics.com/blog/reservation-price-io-insights
48. Floor Price Optimization in Programmatic Advertising Using Machine Learning - TensorOps, accessed July 30, 2025, https://www.tensorops.ai/post/advanced-floor-price-optimization-in-programmatic-advertising-using-ml
49. Real estate decision-making: precision in price prediction through advanced machine learning algorithms | International Journal of Housing Markets and Analysis - Emerald Insight, accessed July 30, 2025, https://www.emerald.com/insight/content/doi/10.1108/ijhma-01-2025-0004/full/html?utm_source=rss&utm_medium=feed&utm_campaign=rss_journalLatest
50. Predicting House Prices (Keras - ANN) - Kaggle, accessed July 30, 2025, https://www.kaggle.com/code/tomasmantero/predicting-house-prices-keras-ann
51. Stacking in Machine Learning - GeeksforGeeks, accessed July 30, 2025, https://www.geeksforgeeks.org/machine-learning/stacking-in-machine-learning/
52. Stacking to Improve Model Performance: A Comprehensive Guide on Ensemble Learning in Python | by Brijesh Soni | Medium, accessed July 30, 2025, https://medium.com/@brijesh_soni/stacking-to-improve-model-performance-a-comprehensive-guide-on-ensemble-learning-in-python-9ed53c93ce28
53. Ensemble Learning: Bagging, Boosting & Stacking - Kaggle, accessed July 30, 2025, https://www.kaggle.com/code/satishgunjal/ensemble-learning-bagging-boosting-stacking
54. Combine predictors using stacking — scikit-learn 1.7.1 documentation, accessed July 30, 2025, https://scikit-learn.org/stable/auto_examples/ensemble/plot_stack_predictors.html
55. Stacking-Based Ensemble Learning Method for House Price Prediction - ResearchGate, accessed July 30, 2025, https://www.researchgate.net/publication/356271517_Stacking-Based_Ensemble_Learning_Method_for_House_Price_Prediction
56. A stacking ensemble model based on Elastic Net and Random Forest for box office prediction - ResearchGate, accessed July 30, 2025, https://www.researchgate.net/publication/382032981_A_stacking_ensemble_model_based_on_Elastic_Net_and_Random_Forest_for_box_office_prediction
57. Understanding SHAP Values: Unlocking Interpretability in Machine ..., accessed July 30, 2025, https://medium.com/@kishanakbari/understanding-shap-values-unlocking-interp

retability-in-machine-learning-ee1f4a9b3a8c
58. Overcoming B2B Pricing Complexity with AI - Pricefx, accessed July 30, 2025, https://www.pricefx.com/learning-center/overcoming-b2b-pricing-complexity-with-ai